3D Solar System Simulation

Project Completion Report

Course: Computer Graphics and Image Processing **Department:** Computer Science and Engineering (CSE) **University:** USTC **Semester:** 7th Semester **Team Size:** 2 Members **Technology:** C++ | OpenGL | GLUT **Status:** ✅ COMPLETED

Table of Contents

---

1. Project Summary

Project Title: 3D Interactive Solar System Simulation

A real-time 3D simulation of the Solar System built using C++ and OpenGL (Legacy/Fixed-Function Pipeline). The simulation renders the Sun, all 8 planets with real surface textures, Earth's Moon, Saturn's ring, an asteroid belt, and a star-filled background — all with proper lighting, Phong shading, orbital mechanics, and interactive camera controls.

The project was built from scratch using only:

- OpenGL (GL/GLU/GLUT) for rendering
- stb_image.h (header-only) for texture loading
- Standard C++17

No game engine, no external framework.

---

2. What Was Planned vs What Was Built

Feature	Planned	Built	Status
Sun at center with glow effect	✅	✅	Done
All 8 planets with unique sizes	✅	✅	Done

Feature	Planned	Built	Status
Real surface textures on planets	✅	✅	Done
Unique orbital speed per planet	✅	✅	Done
Unique self-rotation per planet	✅	✅	Done
Earth's Moon orbiting Earth	✅	✅	Done
Saturn's ring	✅	✅	Done
Asteroid belt (Mars–Jupiter)	✅	✅	Done
Star background	✅	✅	Done
Visible orbit paths	✅	✅	Done
Mouse drag camera rotation	✅	✅	Done
Scroll wheel zoom	✅	✅	Done
Pause / Resume (SPACE)	✅	✅	Done
Speed control (+/-)	✅	✅	Done
Click planet → info panel	✅	✅	Done
Toggle orbit lines (O)	✅	✅	Done
Toggle planet labels (L)	✅	✅	Done
Reset camera (R)	✅	✅	Done
Phong shading (ambient+diffuse+specular)	✅	✅	Done
Sun as point light source (GL_LIGHT0)	✅	✅	Done
Z-buffering / depth test	✅	✅	Done
On-screen HUD (controls help)	✅	✅	Done
FPS target ~60	✅	✅	Done (16ms timer)

Result: Every planned feature was successfully implemented.

---

3. Final Folder Structure

```
Graphics_Project/
│
├── main.cpp                  # Entry point, GLUT setup, game loop, input
│
├── src/
│   ├── planet.cpp / .h       # Planet struct & data for all 8 planets
│   ├── renderer.cpp / .h     # All drawing functions (sun, planets, stars,
etc.)
```

```
│   ├── camera.cpp / .h       # Camera rotation, zoom, reset
│   ├── lighting.cpp / .h     # OpenGL lighting setup (Sun as GL_LIGHT0)
│   ├── texture.cpp / .h      # stb_image texture loader
│   └── ui.cpp / .h           # HUD, planet labels, info panel
│
├── include/
│   └── stb_image.h           # Header-only image loading library
│
├── textures/
│   ├── sun.jpg
│   ├── mercury.jpg
│   ├── venus.jpg
│   ├── earth.jpg
│   ├── mars.jpg
│   ├── jupiter.jpg
│   ├── saturn.jpg
│   ├── uranus.jpg
│   ├── neptune.jpg
│   └── moon.jpg
│
├── Makefile                  # Build configuration
├── solar_system              # Compiled executable
└── PROJECT_DOCUMENTATION.md # This file
```

---

4. Module Implementation Details

main.cpp

- Initializes GLUT window (1200×700, double buffered, depth enabled)
- Registers all callbacks: display, reshape, keyboard, mouse, motion, timer
- Timer fires every 16ms (~60 FPS) to update planet positions
- Handles keyboard input: SPACE, +/-, O, L, R, W, S, ESC
- Handles mouse click for planet selection (screen-space distance picking)

planet.cpp / planet.h

Defines the `Planet` struct:

```
struct Planet {
    string name;
    float radius, orbitRadius;
    float orbitSpeed, rotationSpeed;
    float orbitAngle, rotationAngle;
    GLuint textureID;
    float r, g, b;
    bool hasMoon, hasRing;
    float moonAngle, moonOrbitRadius, moonOrbitSpeed;
};
```

All 8 planets initialized with scientifically inspired relative values.

renderer.cpp / renderer.h

- `drawStars()` — 1500 randomly pre-generated point stars on a sphere of radius ~90
- `drawSun()` — 3 layered blended glow spheres + solid yellow-white core
- `drawOrbit(radius)` — GL_LINE_LOOP circle on XZ plane
- `drawSaturnRing(inner, outer)` — GL_TRIANGLE_STRIP with alpha blending
- `drawAsteroidBelt()` — 300 small random spheres between radius 16.5–18.3
- `drawMoon(idx)` — grey sphere orbiting Earth
- `drawPlanet(idx)` — textured sphere with Phong material, calls ring/moon if needed
- `renderScene()` — calls all of the above in correct order

camera.cpp / camera.h

- Stores `angleX`, `angleY`, `distance`
- Mouse left-drag → updates angles
- Scroll (button 3/4) → adjusts distance
- `applyCamera()` → converts spherical coords to `gluLookAt()` call
- `resetCamera()` → restores default view (angleX=30, distance=55)

lighting.cpp / lighting.h

- `GL_LIGHT0` placed at origin (Sun's position)
- Ambient: very low (0.08) — space is dark
- Diffuse: high (1.0) — Sun is bright
- Specular: medium (1.0) — shiny planet surfaces
- `glEnable(GL_LIGHTING)`, `glEnable(GL_DEPTH_TEST)`, `glEnable(GL_NORMALIZE)`

texture.cpp / texture.h

- Uses `stb_image.h` to load JPG files
- Supports both RGB and RGBA formats
- `GL_LINEAR` filtering for smooth texture display
- Falls back gracefully (solid color) if texture file not found

ui.cpp / ui.h

- `drawHUD()` — top-left controls help text in 2D orthographic overlay
- `drawText()` — renders string using `glutBitmapCharacter`
- `drawInfoPanel(idx)` — semi-transparent blue panel (bottom-right) showing planet name, distance in AU, orbital period
- Planet labels projected from 3D world space to 2D screen using `gluProject()`

---

5. CG Topics Implemented

CG Topic	Implementation
3D Transformations	`glTranslatef`, `glRotatef` for orbit + self-rotation
Perspective Projection	`gluPerspective(45°)` in reshape callback
Viewing / Camera	`gluLookAt()` with spherical coordinate camera
Lighting Model	`GL_LIGHT0` at Sun origin — point light source
Phong Shading	`GL_AMBIENT` + `GL_DIFFUSE` + `GL_SPECULAR` + `GL_SHININESS` per planet
Texture Mapping	`glTexImage2D` + `gluQuadricTexture` on all planets
Animation	Timer-based frame loop, angle incremented per frame
Z-Buffering	`GL_DEPTH_TEST` enabled, cleared each frame
Alpha Blending	Saturn ring + Sun glow using `GL_BLEND`
Geometric Primitives	`gluSphere`, `GL_LINE_LOOP`, `GL_TRIANGLE_STRIP`, `GL_POINTS`
Coordinate Systems	World space → object space → camera space pipeline
Orthographic Projection	2D HUD overlay using `gluOrtho2D`

---

6. Controls Reference

Key / Action	Function
Mouse Left Drag	Rotate camera around solar system
Scroll Up / W	Zoom In
Scroll Down / S	Zoom Out
SPACE	Pause / Resume animation
+	Increase simulation speed
-	Decrease simulation speed
O	Toggle orbit lines ON/OFF
L	Toggle planet labels ON/OFF
R	Reset camera to default view
Left Click on Planet	Show planet info panel
ESC	Exit application

---

7. How to Build & Run

Dependencies (already installed on this system)

```
sudo apt-get install freeglut3-dev libgl1-mesa-dev libglu1-mesa-dev
```

Build

```
cd Graphics_Project
make
```

Run

```
./solar_system
```

Clean

```
make clean
```

Manual compile (without Makefile)

```
g++ main.cpp src/*.cpp -o solar_system -lGL -lGLU -lglut -std=c++17 -I.
```

---

8. Planet Data Reference

Planet	Radius	Orbit Radius	Orbit Speed	Has Moon	Has Ring
Mercury	0.4	4.0	4.74	No	No
Venus	0.9	7.0	1.86	No	No
Earth	1.0	10.0	1.00	Yes	No
Mars	0.5	14.0	0.53	No	No
Jupiter	2.5	20.0	0.08	No	No
Saturn	2.0	27.0	0.03	No	Yes
Uranus	1.5	33.0	0.01	No	No
Neptune	1.4	38.0	0.006	No	No

Speeds are relative — Mercury is fastest (closest to Sun), Neptune is slowest (farthest).

---

9. Known Limitations

- Uses OpenGL Legacy (Fixed-Function Pipeline) — not modern OpenGL 3.3+ with shaders. This was intentional as the course covers classic CG concepts.
- Planet sizes and distances are not to true astronomical scale — they are adjusted for visual clarity.
- Moon texture not applied (grey color used) — moon.jpg is downloaded but not yet mapped.
- No shadow casting between planets (OpenGL fixed pipeline limitation).
- Star brightness flickers slightly due to per-frame random color — cosmetic only.

---

10. References

1. OpenGL Programming Guide (Red Book) — 8th Edition
2. Computer Graphics with OpenGL — Donald Hearn & M. Pauline Baker
3. learnopengl.com — OpenGL tutorials
4. NASA Solar System Exploration — https://solarsystem.nasa.gov
5. Planet Textures — https://www.solarsystemscope.com/textures/
6. stb_image.h — https://github.com/nothings/stb
7. OpenGL GLU Reference — https://www.opengl.org/resources/libraries/glut/

Final Note

> This project successfully implements a complete, real-time 3D Solar System simulation covering all major Computer Graphics course topics in a single application. Every feature listed in the initial project plan was implemented and tested. The simulation runs at ~60 FPS with textures, lighting, shading, animation, and full user interaction.

Completion Report — CSE Department, 7th Semester, USTC Course: Computer Graphics and Image Processing